WEEK 09

Why NoSQL Models Exist and What JSON Enables

Compare data models by workload, then express related CSV data as more than one valid JSON design.

09

Multiple database models exist because workloads disagree

The easiest representation for one question can make another question expensive or risky.

SHAPE

Rows, documents, key-value entries, graphs, and vectors expose different structure.

ACCESS

Exact lookup, relationship traversal, aggregation, and similarity need different paths.

OPERATIONS

Updates, distribution, consistency, indexing, and recovery create different costs.

NoSQL evolved through several overlapping pressures

The label collected different systems; it did not define one shared data model.

1

1970s

Relational theory separates logical questions from physical storage

2

1998

Carlo Strozzi uses NoSQL for a non-SQL relational system

3

2000s

Web-scale workloads drive Bigtable, Dynamo, and distributed design

4

2009

NoSQL becomes a broad community label for nonrelational stores

5

Today

Cloud, multi-model, streaming, search, and vector systems overlap

Historical anchors: Codd; Strozzi NoSQL; Google Bigtable; Amazon Dynamo; see Module 9 sources.

Relational strengths remain valuable in a multi-model world

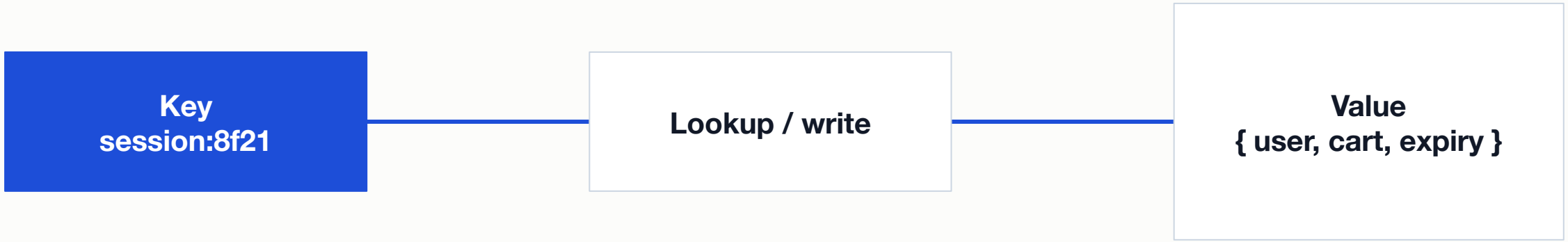
Relational model excels when

- Facts and relationships benefit from explicit constraints
- Many queries combine data in changing ways
- Transactions protect coordinated updates across relations

Another model may help when

- The dominant access path is a direct key or nested aggregate
- Relationship traversal or similarity is the primary question
- Distribution and evolution requirements favor a specialized representation

Key-value stores optimize access through a known key



The application knows the key; the value may be opaque or only partly queryable to the store.

Wide-column systems organize sparse rows by access pattern

Rows can hold different columns, but partition and clustering choices still create a schema.

PARTITION KEY

Determines placement and the unit of distributed access.

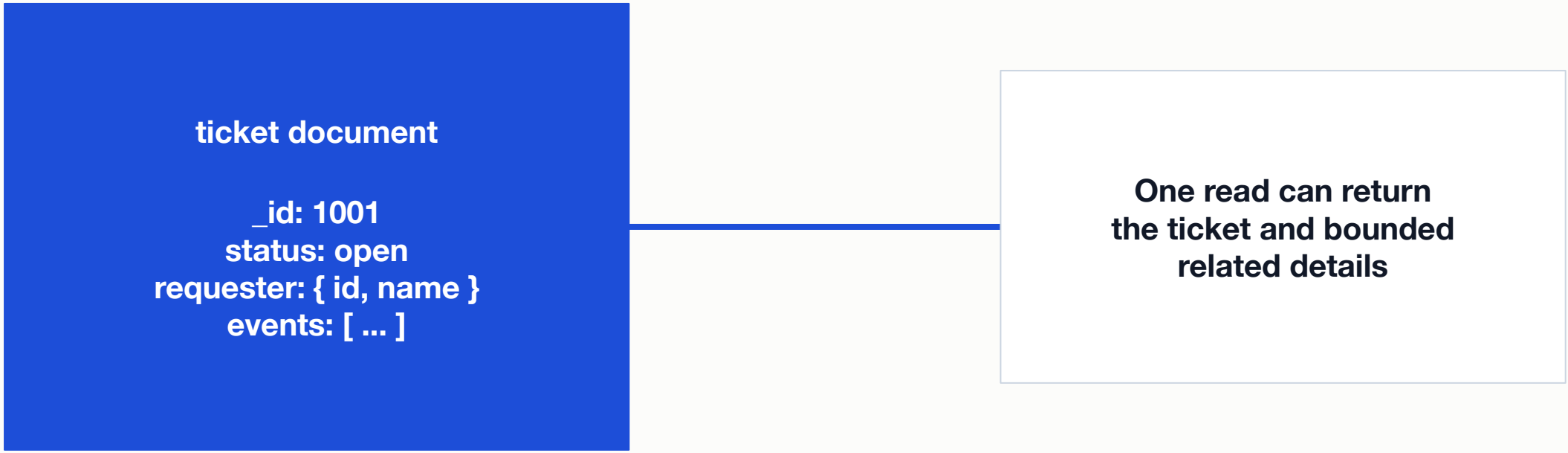
CLUSTERING

Orders related rows within a partition for range access.

TRADEOFF

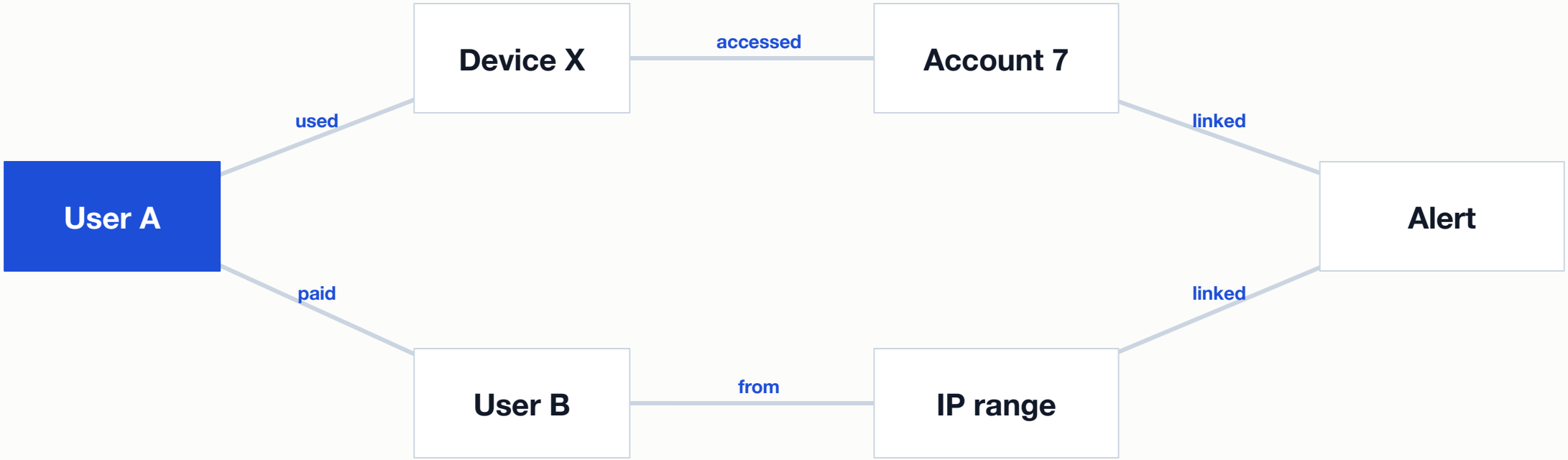
Queries outside the designed access pattern may require another table or scan.

Document databases make nested aggregates first-class



Embedding improves read locality when related data is bounded and commonly read together.

Graph databases make paths and neighborhoods explicit



Vertices are entities; edges are relationships; a path is a sequence of adjacent edges.

Vector databases retrieve nearby representations

$$\text{cosine}(a, b) = (a \cdot b) / (\|a\| \|b\|)$$

VECTOR

An ordered numeric representation, often produced by an embedding model.

SIMILARITY

Cosine compares direction; larger cosine means smaller angle.

INDEX

Approximate nearest-neighbor methods trade exactness for speed and scale.

Cosine and Euclidean measures answer different questions

Cosine similarity

- Compares direction after accounting for magnitude
- Useful when orientation carries semantic meaning
- Requires consistent handling of zero and normalization

Euclidean distance

- Measures straight-line distance between points
- Changes with both direction and magnitude
- On unit vectors, squared distance equals $2 - 2 \times$ cosine

Choose a model from the dominant workload question

Model	Natural question	Primary strength	Common cost
Relational	Which facts match across constrained relations?	integrity + flexible declarative queries	joins and object-relational mapping
Key-value	What value belongs to this exact key?	simple direct lookup and partitioning	weak unknown-key queries
Wide-column	Which ordered rows belong to this partition?	distributed write/read pattern	query-driven duplication
Document	What bounded aggregate is read together?	nested application-shaped record	duplication and growth decisions
Graph / vector	Which path or nearest neighbors matter?	specialized traversal or similarity	specialized operation and evaluation

DAY 2

JSON is strict syntax with flexible composition

Objects and arrays can express many shapes, but a valid shape still needs identity, meaning, and rules.

JSON became popular at the boundary between systems

A small text grammar traveled well across browsers, APIs, languages, documents, and cloud services.

1

JavaScript roots

Familiar object and array notation

2

Early web APIs

Lighter interchange than many XML workflows

3

Language support

Parsers and serializers spread across ecosystems

4

Standards

ECMA-404 and RFC 8259 define the grammar

5

Modern use

APIs, configuration, logs, documents, and events

JSON grammar: ECMA-404 and RFC 8259; historical context: JSON.org and Module 9 sources.

Valid JSON has seven value forms and exact punctuation

```
{
  "ticket_id": 1001,
  "subject": "Streetlight dark",
  "urgent": false,
  "assignee": null,
  "requester": {
    "user_id": 101,
    "name": "Maya Chen"
  },
  "tags": ["streetlight", "safety"]
}
```

Seven forms

- object, array, string, number
- true, false, null
- Double quotes; no comments or trailing comma

One CSV relationship can have multiple valid JSON designs

Embedded item events

- One item object contains an events array
- Easy to read an item with bounded recent history
- Shared or unbounded events can make updates and growth harder

Referenced item and events

- Items and events remain separate arrays or documents
- One event has one source of truth and independent growth
- Reading an item history requires matching the reference

Atlas and GitHub setup are support, not extra deliverables

1. Use GitHub's browser editor for JSON or Markdown when local Git is unavailable.
2. Create only a free Atlas deployment, a database user, and temporary necessary network access.
3. Keep the Atlas URI, password, project details, and IP information out of files and screenshots.
4. Use the text-only JSON lab and validator even if an account or platform is unavailable.

Lab: Turn two CSV tables into multiple JSON designs

Represent Mini Inventory items and events as JSON without running SQL or MQL, then defend one design.

- Inspect identifiers, relationships, cardinality, nulls, and the question the data should answer.
- Write two valid JSON representations, such as embedded and referenced designs.
- Validate both and explain one easier read, one harder update, and one growth assumption.

SUBMIT ONE THING / one text response containing two JSON examples and a comparison

Model choice begins with the question and ends with operating tradeoffs

NoSQL is a family of choices. JSON is a syntax. Neither removes the need to design and verify.

- Which question is natural in this model?
- Which update or growth pattern becomes harder?
- Which identity and relationship must the representation preserve?